

Практическая работа 3.

Введение в Apache Spark: RDD, DataFrame, Spark SQL

Цель работы:

Освоить базовые операции обработки больших данных с использованием Apache Spark (PySpark) на примере агро-данных: работа с RDD, DataFrame API и Spark SQL. Научиться генерировать синтетический датасет показаний сенсоров полей, выполнять трансформации и агрегации, регистрировать временные представления и выполнять SQL-запросы поверх распределённых данных.

Задачи практической работы:

1. Создать SparkSession и выполнить простые операции над RDD (map, filter, reduce).
2. Сгенерировать синтетический агро-датасет показаний сенсоров полей и сохранить его в CSV.
3. Загрузить CSV-файл в Spark DataFrame и выполнить базовые трансформации.
4. Выполнить groupBy-агрегации с использованием DataFrame API (витрина «поле–день»).
5. Зарегистрировать временное представление (temporary view) и выполнить SQL-запросы.
6. Сравнить подходы DataFrame API и Spark SQL по выразительности, читаемости и производительности.
7. Подготовить отчёт с кодом, результатами выполнения и выводами.

1. Подготовка окружения

1.1. Установка Apache Spark (локально).

При наличии локальной среды достаточно установить PySpark:

```
pip install pyspark pandas numpy
```

Проверка:

```
import pyspark
print(pyspark.__version__)
```

1.2. Запуск PySpark в Google Colab.

Если локального Spark нет, используем Google Colab.

В первой ячейке ноутбука:

```
!apt-get install -y openjdk-8-jdk-headless -qq > /dev/null
!pip install -q pyspark findspark pandas numpy

import os, findspark
os.environ["JAVA_HOME"] = "/usr/lib/jvm/java-8-openjdk-amd64"
findspark.init()

from pyspark.sql import SparkSession

spark = (
    SparkSession
    .builder
    .appName("Practicum4_Spark_Agro")
    .master("local[*]") # локальный режим, все ядра
    .config("spark.sql.shuffle.partitions", "4")
    .getOrCreate()
)

print(f"Spark версия: {spark.version}")
print(f"Spark master: {spark.sparkContext.master}")
```

SparkSession — точка входа для работы с Spark.

2. Работа с RDD (Resilient Distributed Datasets).

2.1. Основные концепции RDD.

RDD — базовая абстракция Spark, представляющая распределённую коллекцию элементов.

Свойства:

Resilient — отказоустойчивость через lineage (граф вычислений).

Distributed — данные логически разделены на партии и могут обрабатываться параллельно.

Immutable — операции создают новые RDD, исходный не изменяется.

Типы операций:

Transformations (ленивые): `map`, `filter`, `flatMap`, `reduceByKey` — создают новые RDD.

Actions (действия): `collect`, `count`, `reduce`, `take` — запускают вычисления и возвращают результат.

2.2. Создание RDD из коллекции Python.

```
sc = spark.sparkContext # SparkContext для RDD
```

```
numbers = list(range(1, 21))
```

```
rdd = sc.parallelize(numbers, numSlices=4)
```

```
print("Количество элементов:", rdd.count())
```

```
print("Первые 5 элементов:", rdd.take(5))
```

```
print("Число партий:", rdd.getNumPartitions())
```

2.3. Операции `map` и `filter`.

Задача: оставить только чётные числа и возвести их в квадрат.

```
even_squares_rdd = (  
    rdd  
    .filter(lambda x: x % 2 == 0)  
    .map(lambda x: x * x)  
)
```

```
result = even_squares_rdd.collect()
print("Квадраты чётных чисел:", result)
```

2.4. Операция reduce.

Задача: посчитать сумму квадратов чётных чисел.

```
from operator import add

sum_even_squares = even_squares_rdd.reduce(add)
print("Сумма квадратов чётных чисел:", sum_even_squares)
```

2.5. Задания для самостоятельной работы (RDD).

2.5.1. Посчитайте количество чисел, делящихся на 3 в диапазоне 1–20 (через filter + count).

2.5.2. Найдите максимальное и минимальное значение в исходном RDD с помощью reduce:

```
max_value = rdd.reduce(lambda a, b: a if a > b else b)
min_value = rdd.reduce(lambda a, b: a if a < b else b)
```

2.5.3. Создайте RDD из списка слов и посчитайте общее количество символов:

```
words = ["spark", "rdd", "dataframe", "sql"]
words_rdd = sc.parallelize(words)
# далее map к длине слова и reduce по сумме
```

3. Генерация синтетического агро-датасета.

3.1. Описание датасета.

Мы моделируем показания сенсоров, установленных на полях.

Колонки:

`sensor_id` — идентификатор сенсора (например, `field_3_s1`);
`timestamp` — время измерения;
`field_id` — идентификатор поля (`field_1`, `field_2`, ...);
`soil_temp` — температура почвы (°C);
`soil_moisture` — влажность почвы (условные %);
`air_temp` — температура воздуха (°C).

3.2. Генерация данных в pandas (ниже основа, которую можно рассматривать в качестве примера).

```
import pandas as pd
import numpy as np

np.random.seed(42)

n_fields = 15
sensors_per_field = 2
days = pd.date_range("2024-04-01", periods=10, freq="D")
freq = "1H"

fields = [f"field_{i}" for i in range(1, n_fields+1)]
rows = []

for field_id in fields:
    for sensor_idx in range(1, sensors_per_field+1):
        sensor_id = f"{field_id}_s{sensor_idx}"
        base_soil_temp = np.random.uniform(8, 18)
        base_air_temp = np.random.uniform(10, 20)
        base_moisture = np.random.uniform(25, 45)

        for day in days:
            times = pd.date_range(day, day + pd.Timedelta("23H"), freq=freq)
```

```

    soil_temp = base_soil_temp + 5*np.sin(2*np.pi*(times.hour/24)) +
np.random.normal(0, 1, size=len(times))
    air_temp = base_air_temp + 8*np.sin(2*np.pi*(times.hour/24)) +
np.random.normal(0, 1.5, size=len(times))
    soil_moist = base_moisture + np.random.normal(0, 3, size=len(times))

    # имитируем "засуху" на части полей
    if field_id in ["field_3", "field_7", "field_12"]:
        soil_moist -= np.random.uniform(5, 12)

    for t, st, sm, at in zip(times, soil_temp, soil_moist, air_temp):
        rows.append({
            "sensor_id": sensor_id,
            "timestamp": t,
            "field_id": field_id,
            "soil_temp": round(st, 1),
            "soil_moisture": round(max(sm, 0), 1),
            "air_temp": round(at, 1)
        })

pdf = pd.DataFrame(rows)
csv_path = "agro_sensors.csv"
pdf.to_csv(csv_path, index=False)

print(f"Датасет создан: {len(pdf)} строк")
print(f"Файл сохранён: {csv_path}")
print(pdf.head(10))

```

Студент дополнительно выводит статистику по датасету (число полей, период, describe по числовым колонкам).

4. Загрузка агро-данных в Spark DataFrame

4.1. Чтение CSV в DataFrame.

```
from pyspark.sql.types import StructType, StructField, StringType, DoubleType,
TimestampType

schema = StructType([
    StructField("sensor_id", StringType(), nullable=False),
    StructField("timestamp", TimestampType(), nullable=False),
    StructField("field_id", StringType(), nullable=False),
    StructField("soil_temp", DoubleType(), nullable=False),
    StructField("soil_moisture", DoubleType(), nullable=False),
    StructField("air_temp", DoubleType(), nullable=False)
])

df = (
    spark.read
    .option("header", "true")
    .schema(schema)
    .csv(csv_path)
)

print("Схема DataFrame:")
df.printSchema()
print("Количество строк:", df.count())
df.show(5, truncate=False)
```

4.2. Базовые операции с DataFrame.

Примеры:

```
from pyspark.sql import functions as F

# Количество полей и сенсоров
n_fields = df.select("field_id").distinct().count()
n_sensors = df.select("sensor_id").distinct().count()

print("Число полей:", n_fields)
print("Число сенсоров:", n_sensors)
```

```
# Диапазон времени
df.select(F.min("timestamp").alias("min_ts"),
          F.max("timestamp").alias("max_ts")).show()

# Описательная статистика по температурам и влажности
df.describe("soil_temp", "soil_moisture", "air_temp").show()
```

5. Трансформации DataFrame

5.1. Фильтрация данных

Примеры задач:

выбрать измерения только по одному полю;

выбрать строки, где `soil_moisture < 15` (крайне низкая влажность).

```
dry_measurements = df.filter(F.col("soil_moisture") < 15)
print("Измерений с очень низкой влажностью:", dry_measurements.count())
dry_measurements.select("field_id", "timestamp", "soil_moisture").show(10,
truncatе=False)
```

5.2. Добавление вычисляемых колонок

Добавим:

колонок `date` (день);

флаг `is_low_moisture` (1, если влажность ниже порога).

```
df2 = df.withColumn("date", F.to_date("timestamp"))

df2 = df2.withColumn(
    "is_low_moisture",
    F.when(F.col("soil_moisture") < 20, F.lit(1)).otherwise(F.lit(0))
)

df2.select("field_id", "timestamp", "soil_moisture", "is_low_moisture").show(10,
truncatе=False)
```


5.3. Задания для самостоятельной работы (трансформации)

5.3.1. Добавьте колонку `temp_diff = air_temp - soil_temp` и посчитайте среднюю разницу температур по каждому полю.

5.3.2. Для каждого поля посчитайте долю измерений с низкой влажностью (`soil_moisture < 20`) и выведите поля с наибольшей долей.

6. Агрегации: витрина «поле–день»

6.1. Построение витрины

Задача: построить витрину `field_id`, `date` с агрегатами:

`measurements` — число измерений;

`avg_soil_temp`, `avg_air_temp`;

`avg_moisture`;

`low_moisture_count` — число измерений с низкой влажностью.

```
agg_df = (  
    df2  
    .groupBy("field_id", "date")  
    .agg(  
        F.count("*").alias("measurements"),  
        F.avg("soil_temp").alias("avg_soil_temp"),  
        F.avg("air_temp").alias("avg_air_temp"),  
        F.avg("soil_moisture").alias("avg_moisture"),  
        F.sum("is_low_moisture").alias("low_moisture_count")  
    )  
    .orderBy("field_id", "date")  
)  
  
agg_df.show(20, truncate=False)
```

6.2. Анализ риска засухи

Определим «рисковый день» поля, если `avg_moisture < 20`.

```
risk_days_df = (  
    agg_df
```

```
.filter(F.col("avg_moisture") < 20)
.groupBy("field_id")
.agg(F.count("*").alias("risk_days"))
.orderBy(F.desc("risk_days"))
)

risk_days_df.show()
```

6.3. Задания для самостоятельной работы (агрегации).

6.3.1. Для каждого поля посчитайте минимальную и максимальную среднюю влажность по дням.

6.3.2. Найдите топ-5 полей с наименьшей средней влажностью (по всей истории).

7. Spark SQL: временные представления.

7.1. Регистрация временных представлений.

```
df2.createOrReplaceTempView("sensor_readings")
agg_df.createOrReplaceTempView("field_day_metrics")
```

7.2. Примеры SQL-запросов.

1. Количество измерений и средняя влажность по полям:

```
SELECT
  field_id,
  COUNT(*) AS total_measurements,
  AVG(soil_moisture) AS avg_moisture
FROM sensor_readings
GROUP BY field_id
ORDER BY avg_moisture
```

В Python:

```
sql1 = """
SELECT
```

```
field_id,  
COUNT(*) AS total_measurements,  
AVG(soil_moisture) AS avg_moisture  
FROM sensor_readings  
GROUP BY field_id  
ORDER BY avg_moisture  
""""  
spark.sql(sql1).show()
```

2. Поля с максимальным числом рискованных дней:

```
sql2 = """"  
SELECT  
field_id,  
COUNT(*) AS risk_days  
FROM field_day_metrics  
WHERE avg_moisture < 20  
GROUP BY field_id  
ORDER BY risk_days DESC  
""""  
spark.sql(sql2).show()
```

3. Топ-5 самых сухих дней для конкретного поля:

```
sql3 = """"  
SELECT  
field_id,  
date,  
avg_moisture  
FROM field_day_metrics  
WHERE field_id = 'field_3'  
ORDER BY avg_moisture ASC  
LIMIT 5  
""""  
spark.sql(sql3).show()
```

7.3. Задания для самостоятельной работы (SQL)

7.3.1. Напишите SQL-запрос, который для каждого поля посчитает:

- число дней,
- среднюю, минимальную и максимальную `avg_moisture`.

7.3.2. Найдите поля, у которых доля рискованных дней (`avg_moisture < 20`) больше 50% всех дней.

8. Сравнение DataFrame API и Spark SQL

8.1. Одна и та же задача двумя способами

Задача: посчитать число рискованных дней по полям.

DataFrame API:

```
df_api_result = (  
    agg_df  
    .filter(F.col("avg_moisture") < 20)  
    .groupBy("field_id")  
    .agg(F.count("*").alias("risk_days"))  
    .orderBy("field_id")  
)  
df_api_result.show()
```

Spark SQL:

```
sql_result = spark.sql("""  
SELECT  
    field_id,  
    COUNT(*) AS risk_days  
FROM field_day_metrics  
WHERE avg_moisture < 20  
GROUP BY field_id  
ORDER BY field_id  
""")  
sql_result.show()
```

8.2. Анализ.

Обсудите:

- какой вариант вам субъективно легче читать и писать;
- в каких случаях DataFrame API удобнее (сложная логика, функция, условия);
- в каких — SQL (быстрый аналитический запрос, привычка к SQL).

9. Производительность и Parquet.

(Можно оставить в сокращённом виде.)

- показать `explain()` для одного запроса;
- сохранить `agg_df` в CSV и Parquet;
- сравнить размер и описать, почему Parquet выгоден для аналитики.

10. Дополнительные задания.

- Word Count на RDD (как в оригинале);
- `join` с дополнительной таблицей полей (например, тип почвы);
- window functions: топ-3 поля по минимальной влажности в каждой подгруппе (если вводится дополнительная группировка).

11. Требования к отчёту.

Отчёт:

1. Введение (цель, кратко про Spark, RDD, DataFrame, SQL).
2. Подготовка окружения (версия Spark, как запускали: локально или Colab).
3. RDD: примеры и ответы на задания 2.5.x.
4. Генерация агро-датасета: описание полей, статистика, первые строки.
5. DataFrame API: загрузка, трансформации, витрина «поле–день», агрегаты.
6. Spark SQL: временные представления и SQL-запросы.

7. Сравнение DataFrame vs SQL на агро-задаче.
8. Заключение: чему научились, как это можно использовать в реальных агро-проектах.